

Assignment 6: Socket Programming

Operating System Lab (Unix)

Problem Statement

Write C programs to demonstrate network communication using sockets in a Unix/Linux environment. The objective is to understand how processes communicate over a network using client-server architecture.

Tasks to Implement

1. Write a program to create a TCP server using socket system calls
2. Write a TCP client program to connect to the server
3. Implement message exchange between client and server
4. Modify the program to handle multiple clients (using `fork()`)
5. Write a UDP client-server program
6. Write a program to transfer a file from client to server
7. Implement a simple echo server (server returns same message sent by client)

Instructions

- Use C programming language
- Compile using `gcc filename.c -o output`
- Execute server first, then client
- Use headers: `sys/socket.h`, `netinet/in.h`, `arpa/inet.h`, `unistd.h`
- Use appropriate ports (avoid restricted ports below 1024)
- Handle errors in all socket calls

Sample Output

Example (TCP):

```
Server: Waiting for connection...
Client connected.
Client: Hello Server
Server: Hello Client
```

Example (Echo Server):

```
Client: Test message
Server: Test message
```

Constraints

- Programs must demonstrate actual network communication
- Ensure proper closing of sockets
- Handle connection errors gracefully
- Avoid hardcoding IP addresses where possible

Evaluation Criteria

- Correct Client-Server Implementation – 40%
- Understanding of Socket Concepts – 20%
- Code Structure and Error Handling – 20%
- Multi-client Handling – 10%
- Documentation / Comments – 10%

Submission Guidelines

- Submit all programs in a single compressed (.zip) file
- Folder name: RollNo_Assignment6
- Include client and server source files
- Include sample outputs/screenshots
- Include a README explaining execution steps

Bonus Tasks (Optional but Recommended)

1. Implement a chat application between multiple clients
2. Build a simple HTTP server (respond with HTML page)
3. Implement concurrent server using threads instead of `fork()`
4. Add logging system for server connections
5. Implement authentication (username/password) in client-server